

PROMPT MESTRE

Geração de PDF no Navegador

Arquitetura técnica: Preview → PDF Final → Histórico → Dedução

1. Objetivo e Escopo

Este documento descreve, de forma genérica e reutilizável, a arquitetura de geração de documentos digitais no próprio navegador do usuário. O foco é o pipeline técnico: como os dados do formulário se transformam em preview, como o preview evolui para o PDF final, como o documento é armazenado no histórico e como ocorre a cobrança de créditos. Não serão citados nomes de serviços, órgãos ou modelos específicos de documento.

2. Arquitetura Geral

Princípio fundamental: toda a rasterização acontece no cliente. O backend só executa funções de validação, auditoria e persistência de metadados. O motor é composto por:

- html2canvas-pro: rasteriza o DOM do template em imagens de alta fidelidade.
- jsPDF: monta o PDF final a partir das imagens rasterizadas, página a página.
- pdf.js: renderiza o preview em um canvas leve, sem depender de iframes pesadas.
- Cache de assets: templates e imagens pesadas são decodificadas uma única vez por sessão.
- Gerenciador de loading: centraliza estados de geração para evitar telas pretas/brancas entre transições.

3. Fluxo de Geração: Preview

3.1 Entrada de dados

O usuário preenche um formulário. Cada campo é validado localmente (máscaras, obrigatoriedade, consistência). Ao concluir, o aplicativo monta um objeto payload contendo os dados do formulário, a imagem de fundo do template e quaisquer imagens adicionais (foto, assinatura, QR Code).

3.2 Montagem do HTML de preview

O preview é um HTML invisível no DOM, posicionado fora da viewport (fixed, left: -9999px). Ele contém: a imagem de fundo do template, campos de texto posicionados por coordenadas, imagens dinâmicas (foto, assinatura) e um QR Code de validação. O objetivo do preview é mostrar ao usuário uma versão aproximada do documento final, com baixo custo de memória.

3.3 Rasterização do preview

O motor captura o HTML invisível usando html2canvas-pro com escala reduzida (geralmente 1.5x a 2x). A saída é uma imagem JPEG comprimida, leve o suficiente para ser exibida

instantaneamente em um canvas. Durante a captura, uma marca d'água semitransparente é sobreposta para indicar que é apenas preview.

3.4 Apresentação ao usuário

O preview é apresentado em um componente de canvas próprio. O componente cria um novo elemento canvas a cada renderização, descartando o anterior, para evitar corrupção de estado gráfico em navegadores móveis. O loading só some quando o primeiro frame do PDF já foi desenhado, garantindo que o usuário nunca veja uma tela em branco.

4. Fluxo de Geração: PDF Final

4.1 Gatilho de geração

Quando o usuário confirma o preview, o sistema verifica saldo de créditos. Se houver saldo, inicia a geração final. A dedução só ocorre APÓS o PDF ser gerado com sucesso, nunca antes. Isso evita cobranças por falhas técnicas.

4.2 HTML de alta resolução

O mesmo HTML do preview é clonado, mas com escala muito maior (alta densidade de pixels, tipicamente ~576 DPI equivalente). O template de fundo é reutilizado do cache de assets para evitar reconversão. O QR Code é regenerado na resolução final, mantendo a mesma posição e dimensões do preview.

4.3 Divisão em bandas adaptativa

Para evitar travamentos em dispositivos com pouca memória, o documento é dividido em bandas horizontais. A altura da banda é adaptativa: se uma banda renderizar em menos de 1.5s, a próxima terá o dobro da altura; se falhar por falta de memória, a altura é reduzida e a renderização é repetida. Cada banda é convertida em JPEG assíncrono via Blob/FileReader para não bloquear a thread principal.

4.4 Montagem do PDF

As bandas são inseridas no jsPDF uma a uma, com as dimensões exatas da página. O PDF resultante é convertido para base64/dataURL e entregue à página de resultado. A qualidade final é máxima, sem marca d'água e sem perda de fidelidade visual.

4.5 Detecção de falhas

Após cada renderização, o motor executa um 'anti-black probe': verifica se a banda ficou totalmente preta ou transparente (pixel alpha < 8). Se detectar, a banda é re-renderizada com menor escala ou maior fragmentação. Isso evita PDFs corrompidos ou em branco.

5. Otimizações de Performance

- Warm-up: os módulos html2canvas-pro, jsPDF e pdf.js são pré-carregados assim que o usuário entra no formulário, não na hora da geração.

- Cache de templates: imagens de fundo pesadas (WebP) são mantidas em blob URLs durante toda a sessão.
- Cache de preview HTML: o payload do preview é hashado; se o usuário voltar sem alterar dados, o preview anterior é reutilizado.
- Orçamento de pixels: o canvas do preview respeita um orçamento baseado na memória do dispositivo (3M/6M/12M pixels), evitando tela preta em iOS/Android.
- Descarte de payloads antigos: apenas o preview mais recente é mantido na memória, evitando duplicação de PDFs e fotos.
- Canvas novo por render: o componente de preview nunca reaproveita o mesmo elemento canvas, evitando transformações corrompidas no WebKit.

6. Histórico e Armazenamento

6.1 Persistência do documento

Após a geração bem-sucedida e a dedução de créditos, o PDF final é salvo no histórico do usuário. O registro contém: nome/identificação do documento, data de referência, tipo, informações adicionais serializadas em JSON, data de criação e o próprio PDF em base64 (ou referência a storage, dependendo da política de privacidade).

6.2 Ações do histórico

No histórico, o usuário pode: visualizar o documento novamente, baixar o PDF, compartilhar via Web Share API, renovar o documento (custo de 1 crédito, estende a validade em +30 dias) e excluir o registro. A exclusão remove o registro do banco e, se aplicável, o arquivo de storage.

6.3 Auditoria e logs

Toda ação sensível (geração, renovação, exclusão, alteração de créditos por staff) gera um log de auditoria. Os logs são enxugados automaticamente: a cada 50 novos registros de um usuário, os 50 mais antigos são removidos, mantendo o banco enxuto sem perder rastreabilidade recente.

7. Dedução de Créditos e Segurança

7.1 Verificação de saldo

Antes de iniciar a geração final, o frontend consulta o saldo do usuário. Se o saldo for insuficiente, a geração é bloqueada e uma mensagem orienta a recarga. O cálculo do custo respeita o plano ativo do usuário, aplicando descontos progressivos conforme a hierarquia do plano.

7.2 Dedução atômica

A dedução é feita por uma RPC no banco (por exemplo, `consume_credits`), que decrementa o saldo e registra a transação em uma única operação atômica. Se a dedução falhar, o PDF gerado não é entregue ao usuário e nenhum crédito é perdido. Se a geração do PDF falhar, a dedução não é chamada.

7.3 Controle de acesso

A geração só é permitida para usuários aprovados e verificados. Usuários recém-criados permanecem em estado 'não verificado' até que um administrador clique em 'Verificar conta'. Enquanto não verificados, podem fazer login, mas não conseguem acessar formulários nem gerar documentos.

7.4 Proteção de dados administrativos

Contas administrativas nunca devem ter e-mail, UUID ou identificadores expostos em tabelas acessíveis por gerentes ou usuários comuns. As políticas de RLS devem filtrar linhas de administradores para qualquer role inferior. RPCs de gerenciamento devem rejeitar tentativas de alterar contas administrativas.

8. Validação Pública

Documentos que precisam de validação externa recebem um QR Code dinâmico. Ao escanear, o visitante é direcionado a uma página pública de validação. Essa página consulta uma Edge Function privada (com chave de API interna) e retorna apenas dados mascarados: nome parcial, tipo do documento, status de validade e data de expiração. Nenhum dado sensível completo é exposto no front público.

9. UX e Estados de Loading

A experiência de geração deve ser fluida. O overlay de loading só aparece se a operação durar mais de 200ms e permanece no mínimo 500ms para evitar piscadas. A mensagem informa explicitamente que o processo pode levar alguns segundos e que a tela não deve ser fechada. Ao concluir, o usuário é levado automaticamente para a tela de resultado, já com o PDF renderizado.

10. Estrutura de Dados Recomendada

Payload de geração:

```
{
  formData: { /* dados do formulário */ },
  templateUrl: 'url-do-template-webp',
  photoBase64: 'data:image/jpeg;base64,...',
  signatureBase64: 'data:image/png;base64,...',
  qrToken: 'uuid-de-validacao',
  preview: true | false
}
```

Registro de histórico:

```
{
  id, user_id, type, name, identification, date,
  description, additional_info (JSON),
  pdf_data_url, created_at, expires_at
}
```

11. Checklist de Implementação

1. Formulário valida todos os campos antes de montar o payload.
2. Preview renderiza em resolução reduzida com marca d'água.
3. PDF final usa a mesma estrutura do preview, mas em alta resolução.
4. Dedução só ocorre após sucesso na geração.
5. Histórico salva metadados + PDF final.
6. Validação pública expõe apenas dados mascarados.
7. Cache de templates e preview evita reprocessamento.
8. Anti-black probe garante integridade visual.
9. Loading overlay sincronizado com o primeiro frame renderizado.
10. RLS e RPCs protegem contas administrativas e limitam ações de gerentes.

12. Considerações Finais

A geração no navegador elimina custos de APIs externas de PDF e permite fidelidade visual absoluta, pois o mesmo HTML usado no preview é reaproveitado no final. O desafio principal é o gerenciamento de memória em dispositivos móveis. Por isso, divisão em bandas, cache inteligente, orçamento de pixels e fallback automático são obrigatórios. Sempre que ajustar a qualidade do preview, isolar a mudança para não afetar o PDF final — e vice-versa.